S03.E1 - EJERCICIO PRACTICO 01 (SESION 03)

Estructuras de Datos Avanzadas (JSONB/Arrays) y Optimización de Consultas

Entorno de Trabajo: demodb (esquema bookings)	**Duración Estimada:** 25 Minutos
Modalidad: Individual o en Parejas	**Recursos de Referencia:** demodb_bookings.md y demodb_esquema.png

Parte 1: Análisis Teórico y Fundamentos (5 minutos)

Cuestión 1.1 (Arrays vs. Normalización 1:N): En la tabla de rutas bookings.routes (alias rt), los días de la semana en que opera cada vuelo se almacenan en la columna days_of_week como un array de enteros (integer[]). Explica brevemente qué ventajas de rendimiento y simplicidad ofrece almacenar colecciones homogéneas fijas mediante un tipo ARRAY frente a crear una tabla relacional dedicada. Indica además qué operador de arrays (&& de solapamiento o @> de contención) se utiliza para filtrar rutas operativas en fin de semana (sábado = 6, domingo = 7).

Cuestión 1.2 (JSONB, Índices GIN y EXPLAIN ANALYZE): En la tabla bookings.airports_data (alias aid), la columna city almacena traducciones multi-idioma en formato jsonb.
a) ¿Por qué la condición aid.city @> '{"en": "London"}'::jsonb puede utilizar un índice invertido **GIN**, mientras que aid.city->>'en' = 'London' requeriría un índice funcional explícito o realizar un Seq Scan?
b) Al auditar una consulta con EXPLAIN (ANALYZE, BUFFERS), ¿cuál es la diferencia conceptual entre el valor de Cost estimado por el optimizador y el valor de Actual Time medido en milisegundos?

Parte 2: Caso Práctico en demodb (20 minutos)

Escenario Operativo: El departamento de ingeniería de datos requiere construir consultas de alto rendimiento aprovechando las estructuras multimodelo existentes en demodb (arrays de días de operación en rutas y documentos jsonb con traducciones de ciudades y modelos), así como diagnosticar la velocidad de ejecución y el consumo de búfer mediante la herramienta EXPLAIN ANALYZE.

Tarea 1: Consulta Multimodelo con Arrays y JSONB (10 minutos)

Escribe una consulta SELECT que combine la información de rutas con los datos maestros traducidos de aeropuertos y aviones en demodb. La consulta debe cumplir los siguientes requisitos exactos de esquema:
• **Tablas y Uniones Reales:**
- Tabla principal: bookings.routes (alias rt).
- INNER JOIN con bookings.airports_data (alias aid_dep) usando rt.departure_airport = aid_dep.airport_code.
- INNER JOIN con bookings.airports_data (alias aid_arr) usando rt.arrival_airport = aid_arr.airport_code.
- INNER JOIN con bookings.airplanes_data (alias apd) usando rt.airplane_code = apd.airplane_code.
• **Campos Extraídos:** Número de ruta (rt.route_no), días operativos (rt.days_of_week), ciudad de salida en inglés (aid_dep.city->>'en' AS ciudad_salida_en), ciudad de llegada en inglés (aid_arr.city->>'en' AS ciudad_llegada_en), y modelo de avión en inglés (apd.model->>'en' AS modelo_avion_en).
• **Filtro de Array:** Aplica el operador de solapamiento de arrays && sobre rt.days_of_week para obtener únicamente rutas que operen en fin de semana (sábado 6 o domingo 7): rt.days_of_week && ARRAY[6, 7].
• **Filtro JSONB:** Utiliza el operador de contención @> sobre aid_dep.city para requerir que la ciudad de salida contenga la clave en inglés: aid_dep.city @> '{"en": "London"}'::jsonb.
• **Orden y Límite:** Ordena el resultado por rt.route_no ascendente y limita a 50 filas.

Tarea 2: Diagnóstico y Optimización con CTE y EXPLAIN (10 minutos)

Diseña una consulta de análisis de rendimiento utilizando una Expresión de Tabla Común (CTE / WITH) para auditar las reservas de alto valor y sus segmentos de vuelo asociados:

1. Define una CTE llamada reservas_altas que extraiga book_ref, book_date, y total_amount de bookings.bookings (alias bk) donde total_amount > 50000.00.
2. Une la CTE reservas_altas (alias ra) con la tabla bookings.tickets (alias tk enlazada por ra.book_ref = tk.book_ref) y con bookings.segments (alias sg enlazada por tk.ticket_no = sg.ticket_no).
3. Muestra ra.book_ref, tk.passenger_name, sg.flight_id, sg.fare_conditions y sg.price.
4. Antepone EXPLAIN (ANALYZE, BUFFERS) a la consulta e interpreta en una breve nota qué tipos de escaneo (Seq Scan o Index Scan) ejecuta el optimizador sobre cada tabla.